Vacation Scheduler – Test Plan and Results

1. Test Plan Overview

Item	Detail
Application	Vacation Scheduler (Android)
Test Type	Unit Tests (JUnit 4)
Test File	`VacationValidationTest.java`
Framework	JUnit 4 via `androidx.test`
Execution	`./gradlew testDebugUnitTest`

---

2. Test Categories and Objectives

Category 1 – Date Format Validation

Verify that `isValidDateFormat()` correctly accepts and rejects date strings.

Test ID	Test Name	Input	Expected Result
T01	`validDate_returnsTrue`	`"06/15/2025"`	true
T02	`invalidDateFormat_returnsFalse`	`"2025-06-15"`	false
T03	`emptyDate_returnsFalse`	`""`	false
T04	`nullDate_returnsFalse`	`null`	false
T05	`impossibleDate_returnsFalse`	`"13/40/2025"`	false

Category 2 – Date Range Validation

Verify that end date / start date ordering is enforced correctly.

Test ID	Test Name	Start	End	Expected Result
T06	`endDateAfterStartDate_returnsTrue`	`06/01/2025`	`06/15/2025`	true
T07	`endDateEqualToStartDate_returnsTrue`	`06/01/2025`	`06/01/2025`	true
T08	`endDateBeforeStartDate_returnsFalse`	`06/15/2025`	`06/01/2025`	false

Category 3 – Excursion Date Boundary Validation

Verify that excursion dates must fall within the vacation period.

Test ID	Test Name	Excursion Date	Vacation Start	Vacation End	Expected Result
T09	`excursionDateWithinVacation_returnsTrue`	`06/10/2025`	`06/01/2025`	`06/30/2025`	true
T10	`excursionDateOnVacationStartDate_returnsTrue`	`06/01/2025`	`06/01/2025`	`06/30/2025`	true
T11	`excursionDateOnVacationEndDate_returnsTrue`	`06/30/2025`	`06/01/2025`	`06/30/2025`	true
T12	`excursionDateBeforeVacation_returnsFalse`	`05/31/2025`	`06/01/2025`	`06/30/2025`	false
T13	`excursionDateAfterVacation_returnsFalse`	`07/01/2025`	`06/01/2025`	`06/30/2025`	false
T14	`excursionDateWithEmptyVacationDates_returnsTrue`	`06/10/2025`	`""`	`""`	true

Category 4 – Input Length Validation

Verify that name and title length constraints are enforced.

Test ID	Test Name	Input	Expected Result
T15	`vacationName_notEmpty_isValid`	`"Hawaii Trip"`	valid (not empty)
T16	`vacationName_empty_isInvalid`	`"  "`	invalid (empty after trim)
T17	`vacationName_withinLengthLimit_isValid`	11-char string	true (≤ 100)

Test ID	Test Name	Input	Expected Result
T18	`vacationName_exceedsLengthLimit_isInvalid`	101-char string	false (> 100)

Category 5 – OOP: Encapsulation

Verify that private fields are accessible only through public getters and setters.

Test ID	Test Name	Object	Expected Result
T19	`vacation_encapsulation_setAndGetId`	Vacation, setId(42)	getId() == 42
T20	`excursion_encapsulation_setAndGetVacationId`	Excursion, setVacationId(99)	getVacationId() == 99

Category 6 – OOP: Polymorphism and Inheritance

Verify that `Vacation` and `Excursion` correctly implement the `ScheduleItem` interface and override its methods polymorphically.

Test ID	Test Name	Expected Result
T21	`vacation_getDisplayTitle_returnsName`	`"Beach Trip"`
T22	`vacation_getDisplayDate_returnsCombinedDates`	`"07/01/2025 - 07/10/2025"`
T23	`excursion_getDisplayTitle_returnsTitle`	`"Snorkeling"`
T24	`excursion_getDisplayDate_returnsDate`	`"07/05/2025"`
T25	`vacation_implementsScheduleItem`	instanceof ScheduleItem == true
T26	`excursion_implementsScheduleItem`	instanceof ScheduleItem == true
T27	`scheduleItem_polymorphicDisplayTitle_vacation`	`"Mountain Hike"` via ScheduleItem reference
T28	`scheduleItem_polymorphicDisplayTitle_excursion`	`"Zip Line"` via ScheduleItem reference

3. Unit Test Scripts

File:
`app/src/test/java/com/example/vacation_sceduler/VacationValidationTest.java`

```java
package com.example.vacation_sceduler;

import org.junit.Before;
import org.junit.Test;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;

import static org.junit.Assert.*;

import com.example.vacation_sceduler.entities.Excursion;
import com.example.vacation_sceduler.entities.ScheduleItem;
import com.example.vacation_sceduler.entities.Vacation;

public class VacationValidationTest {

    private static final String DATE_FORMAT = "MM/dd/yyyy";
    private SimpleDateFormat dateFormat;

    @Before
    public void setUp() {
        dateFormat = new SimpleDateFormat(DATE_FORMAT, Locale.US);
        dateFormat.setLenient(false);
    }

    // --- Helper: Date Format ---
    private boolean isValidDateFormat(String dateStr) {
        if (dateStr == null || dateStr.isEmpty()) return false;
        try {
            dateFormat.parse(dateStr);
            return true;
        } catch (ParseException e) {
            return false;
        }
    }

    // --- Helper: Date Range ---
    private boolean isEndDateAfterStartDate(String startStr, String endStr) {
        try {
            Date start = dateFormat.parse(startStr);
            Date end = dateFormat.parse(endStr);
            return end != null && start != null && !end.before(start);
        } catch (ParseException e) {
            return false;
        }
    }
```

```java
    // --- Helper: Excursion Boundary ---
    private boolean isDateWithinVacation(String excursionDateStr, String vaca
tionStart, String vacationEnd) {
        if (vacationStart == null || vacationStart.isEmpty() ||
                vacationEnd == null || vacationEnd.isEmpty()) return true;
        try {
            Date excursionDate = dateFormat.parse(excursionDateStr);
            Date startDate = dateFormat.parse(vacationStart);
            Date endDate = dateFormat.parse(vacationEnd);
            return excursionDate != null && startDate != null && endDate != n
ull
                    && !excursionDate.before(startDate) && !excursionDate.aft
er(endDate);
        } catch (ParseException e) {
            return true;
        }
    }

    // ==============================================
    // Category 1: Date Format Validation (T01-T05)
    // ==============================================

    @Test
    public void validDate_returnsTrue() {
        assertTrue(isValidDateFormat("06/15/2025"));
    }

    @Test
    public void invalidDateFormat_returnsFalse() {
        assertFalse(isValidDateFormat("2025-06-15"));
    }

    @Test
    public void emptyDate_returnsFalse() {
        assertFalse(isValidDateFormat(""));
    }

    @Test
    public void nullDate_returnsFalse() {
        assertFalse(isValidDateFormat(null));
    }

    @Test
    public void impossibleDate_returnsFalse() {
        assertFalse(isValidDateFormat("13/40/2025"));
    }

    // ==============================================
    // Category 2: Date Range Validation (T06-T08)
```

```
    // ==========================================

    @Test
    public void endDateAfterStartDate_returnsTrue() {
        assertTrue(isEndDateAfterStartDate("06/01/2025", "06/15/2025"));
    }

    @Test
    public void endDateEqualToStartDate_returnsTrue() {
        assertTrue(isEndDateAfterStartDate("06/01/2025", "06/01/2025"));
    }

    @Test
    public void endDateBeforeStartDate_returnsFalse() {
        assertFalse(isEndDateAfterStartDate("06/15/2025", "06/01/2025"));
    }

    // ====================================================
    // Category 3: Excursion Date Boundary Validation (T09–T14)
    // ====================================================

    @Test
    public void excursionDateWithinVacation_returnsTrue() {
        assertTrue(isDateWithinVacation("06/10/2025", "06/01/2025", "06/30/20
25"));
    }

    @Test
    public void excursionDateOnVacationStartDate_returnsTrue() {
        assertTrue(isDateWithinVacation("06/01/2025", "06/01/2025", "06/30/20
25"));
    }

    @Test
    public void excursionDateOnVacationEndDate_returnsTrue() {
        assertTrue(isDateWithinVacation("06/30/2025", "06/01/2025", "06/30/20
25"));
    }

    @Test
    public void excursionDateBeforeVacation_returnsFalse() {
        assertFalse(isDateWithinVacation("05/31/2025", "06/01/2025", "06/30/2
025"));
    }

    @Test
    public void excursionDateAfterVacation_returnsFalse() {
        assertFalse(isDateWithinVacation("07/01/2025", "06/01/2025", "06/30/2
025"));
```

```java
    }

    @Test
    public void excursionDateWithEmptyVacationDates_returnsTrue() {
        assertTrue(isDateWithinVacation("06/10/2025", "", ""));
    }

    // ==============================================
    // Category 4: Input Length Validation (T15–T18)
    // ==============================================

    @Test
    public void vacationName_notEmpty_isValid() {
        String name = "Hawaii Trip";
        assertFalse(name.trim().isEmpty());
    }

    @Test
    public void vacationName_empty_isInvalid() {
        String name = "   ";
        assertTrue(name.trim().isEmpty());
    }

    @Test
    public void vacationName_withinLengthLimit_isValid() {
        String name = "Hawaii Trip";
        assertTrue(name.length() <= 100);
    }

    @Test
    public void vacationName_exceedsLengthLimit_isInvalid() {
        String name = "A".repeat(101);
        assertFalse(name.length() <= 100);
    }

    // ==============================================
    // Category 5: OOP – Encapsulation (T19–T20)
    // ==============================================

    @Test
    public void vacation_encapsulation_setAndGetId() {
        Vacation vacation = new Vacation("Test", "Hotel", "01/01/2025", "01/1
0/2025");
        vacation.setId(42);
        assertEquals(42, vacation.getId());
    }

    @Test
    public void excursion_encapsulation_setAndGetVacationId() {
```

```java
        Excursion excursion = new Excursion("Tour", "01/05/2025", 1);
        excursion.setVacationId(99);
        assertEquals(99, excursion.getVacationId());
    }

    // ========================================================
    // Category 6: OOP – Polymorphism and Inheritance (T21–T28)
    // ========================================================

    @Test
    public void vacation_getDisplayTitle_returnsName() {
        Vacation vacation = new Vacation("Beach Trip", "Hilton", "07/01/2025"
, "07/10/2025");
        assertEquals("Beach Trip", vacation.getDisplayTitle());
    }

    @Test
    public void vacation_getDisplayDate_returnsCombinedDates() {
        Vacation vacation = new Vacation("Beach Trip", "Hilton", "07/01/2025"
, "07/10/2025");
        assertEquals("07/01/2025 - 07/10/2025", vacation.getDisplayDate());
    }

    @Test
    public void excursion_getDisplayTitle_returnsTitle() {
        Excursion excursion = new Excursion("Snorkeling", "07/05/2025", 1);
        assertEquals("Snorkeling", excursion.getDisplayTitle());
    }

    @Test
    public void excursion_getDisplayDate_returnsDate() {
        Excursion excursion = new Excursion("Snorkeling", "07/05/2025", 1);
        assertEquals("07/05/2025", excursion.getDisplayDate());
    }

    @Test
    public void vacation_implementsScheduleItem() {
        Vacation vacation = new Vacation("Test", "Hotel", "01/01/2025", "01/1
0/2025");
        assertTrue(vacation instanceof ScheduleItem);
    }

    @Test
    public void excursion_implementsScheduleItem() {
        Excursion excursion = new Excursion("Test", "01/05/2025", 1);
        assertTrue(excursion instanceof ScheduleItem);
    }

    @Test
```

```java
    public void scheduleItem_polymorphicDisplayTitle_vacation() {
        ScheduleItem item = new Vacation("Mountain Hike", "Lodge", "08/01/202
5", "08/07/2025");
        assertEquals("Mountain Hike", item.getDisplayTitle());
    }

    @Test
    public void scheduleItem_polymorphicDisplayTitle_excursion() {
        ScheduleItem item = new Excursion("Zip Line", "08/03/2025", 2);
        assertEquals("Zip Line", item.getDisplayTitle());
    }
}
```

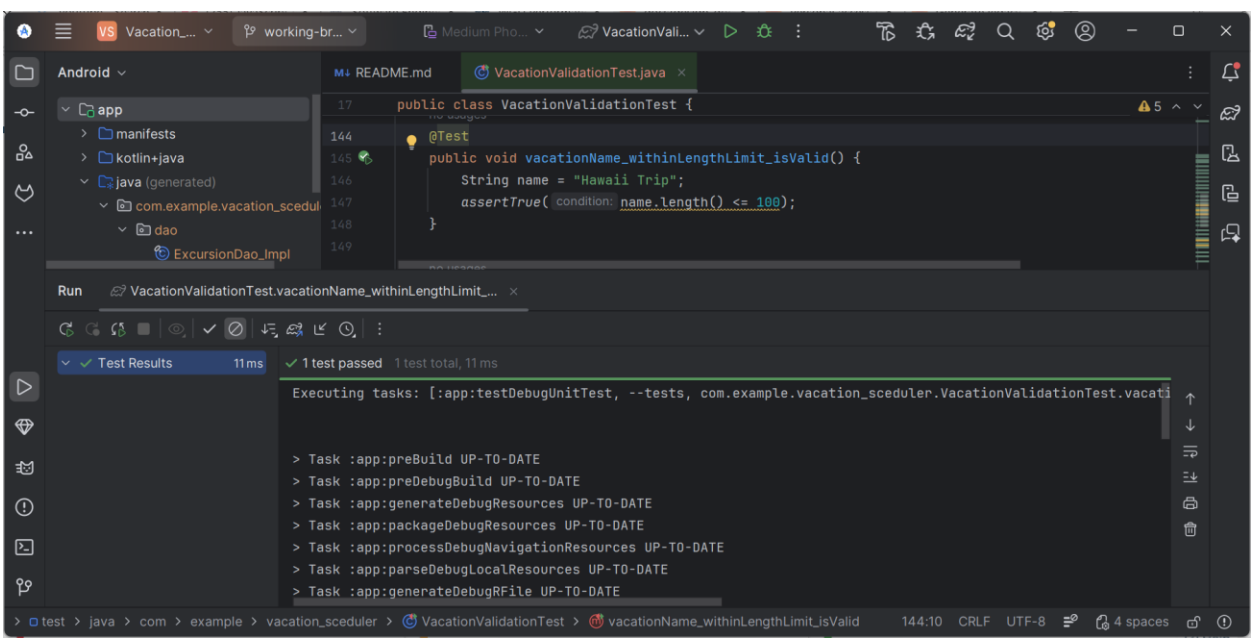
---

4. Test Results

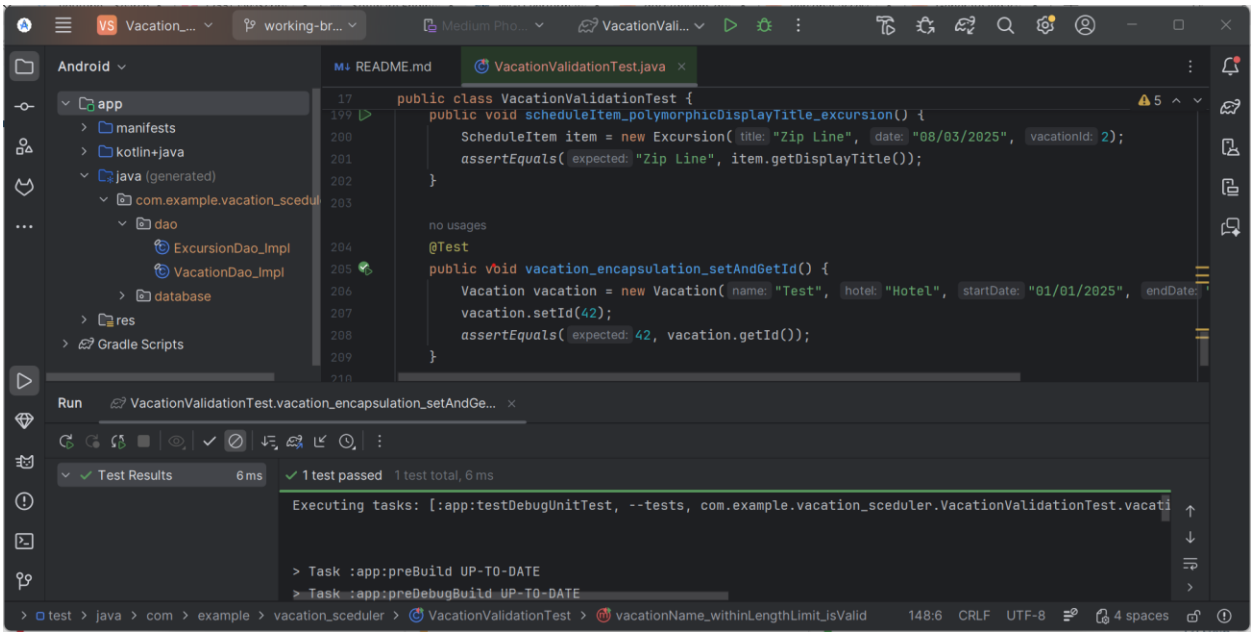
Total Tests: 28 | Passed: 28 | Failed: 0 | Skipped: 0

All 28 unit tests passed on the first run after fixes were applied.

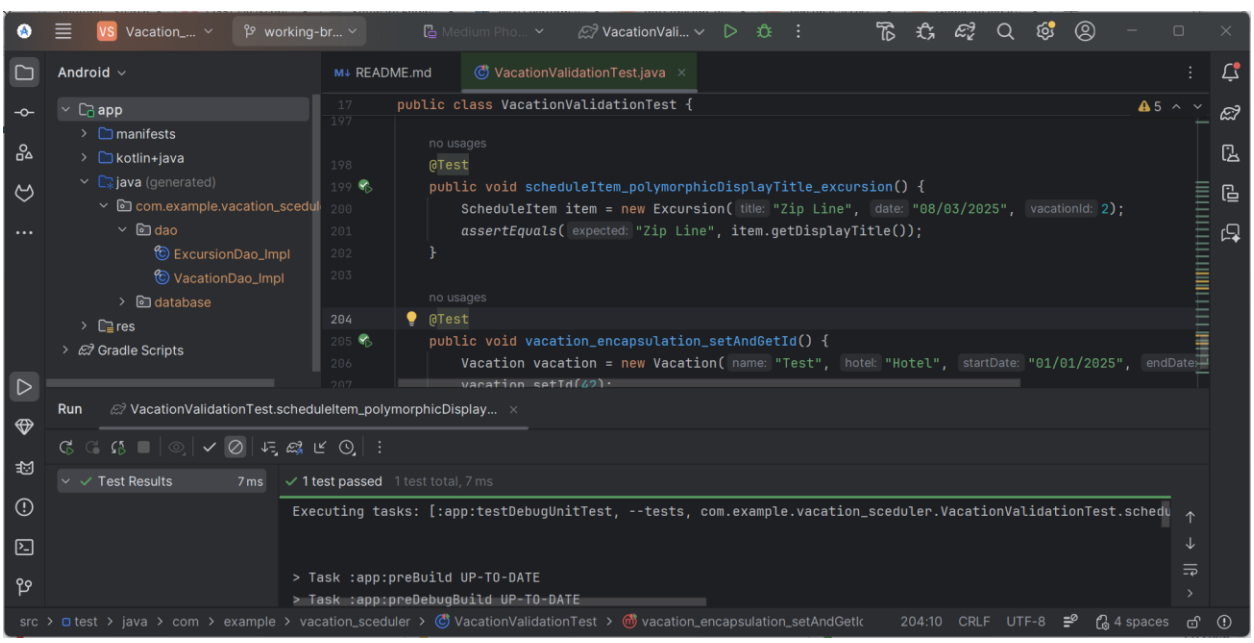
Screenshot 1 – Android Studio: Individual test run (T17 – vacationName_withinLengthLimit_isValid)



Screenshot 2 – Android Studio: Individual test run (T19 – vacation_encapsulation_setAndGetId)



Screenshot 3 – Android Studio: Individual test run (T27 – scheduleItem_polymorphicDisplayTitle_excursion)



5. Summaries of Changes Resulting from Completed Tests

During the implementation of tests, the following issues were identified and resolved:

#	Issue Found	File Affected	Change Made
1	`build.gradle.kts` root file had a comment and `plugins {` merged onto the same line, breaking Gradle compilation	`build.gradle.kts`	Fixed by separating the comment from the `plugins {` block onto its own line
2	`settings.gradle.kts` was missing from the project root, preventing command-line Gradle execution	`settings.gradle.kts`	Created `settings.gradle.kts` with correct `pluginManagement` and `dependencyResolutionManagement` repository configuration
3	`isEndDateAfterStartDate()` had an unchecked potential null dereference on parsed dates	`VacationDetailActivity.java`	Added null checks before calling `.before()` on parsed `Date` objects
4	`setExcursionAlert()` had an unchecked null dereference on the parsed excursion date	`ExcursionDetailActivity.java`	Wrapped date usage in a null check before calling `setAlert()`